

Nota de referência

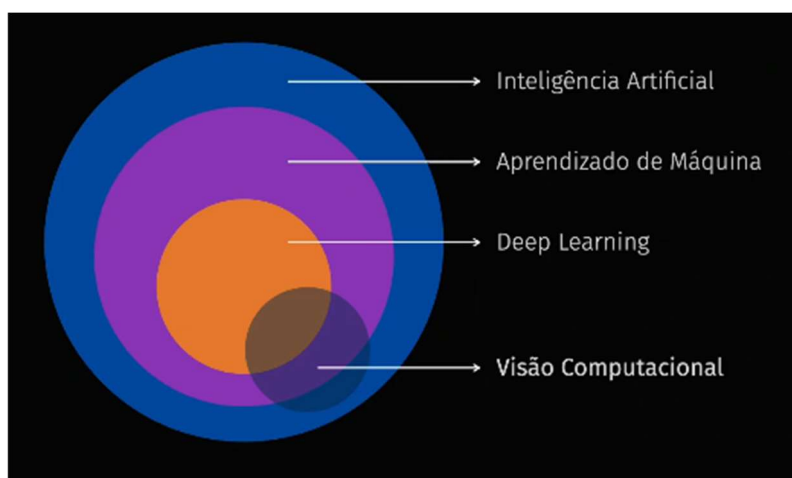
Este material reúne minhas anotações de estudo desenvolvidas ao longo do curso de Visão Computacional da NexVisual, realizado em parceria com o Instituto Eldorado.

O conteúdo aqui apresentado foi organizado com fins educacionais e de revisão pessoal, a partir das aulas, explicações, atividades práticas e exemplos trabalhados durante o curso. As anotações refletem meu processo de aprendizagem, podendo conter adaptações, complementações, interpretações próprias e exemplos desenvolvidos durante os estudos.

Agradeço à NexVisual e ao Instituto Eldorado pela oportunidade de aprendizado e pela contribuição para minha formação na área de Visão Computacional.

Aula 1 – O que é visão computacional?

É uma área inteligência artificial e sua principal função é criar algoritmos e modelos que consigam extrair informações de forma organizadas a partir de dados visuais, como imagens e vídeos.



O que é uma imagem e vídeo do ponto de vista técnico?

Uma imagem é uma estrutura de dados do registro visual de uma cena em um instante de tempo. Já um vídeo é uma sequência de imagem ao longo do tempo. A imagem mostra o “agora” o vídeo o durante.

Exemplos de algoritmos aplicados:

- Exploração de planetas para mapeamento de terrenos e planejamento de movimentação dos robôs Rover (NASA);

- Sistema de vigilância para detecção de pessoas ou animais;
- Análise de exames médicos.

Problemas clássicos da visão computacional:

Classificação, detecção e segmentação de imagens que serão abordados a seguir.

Aula 2 – Manipulação de arrays com NumPy

O que é uma imagem em um mundo digital?

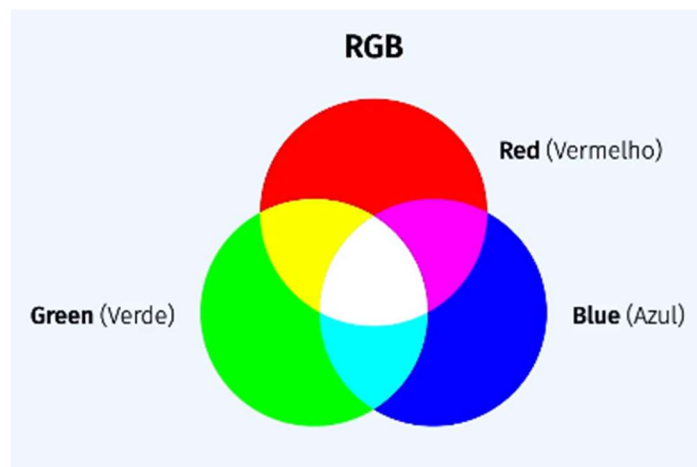
Uma imagem digital é um conjunto de pixels.

O que é um pixel?

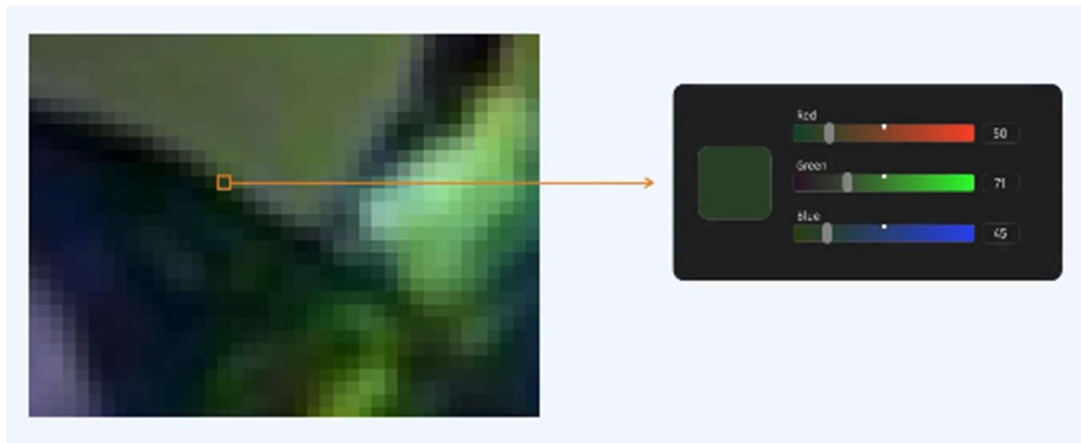
É a menor unidade de uma imagem, cada pixel carrega informações de cor e brilho.

O pixel pode ser representado em escala de cores preto e branco e nesse caso sua representação de cor várias apenas na intensidade luminosa (equivalendo a quantidade de “luz” do pixel).

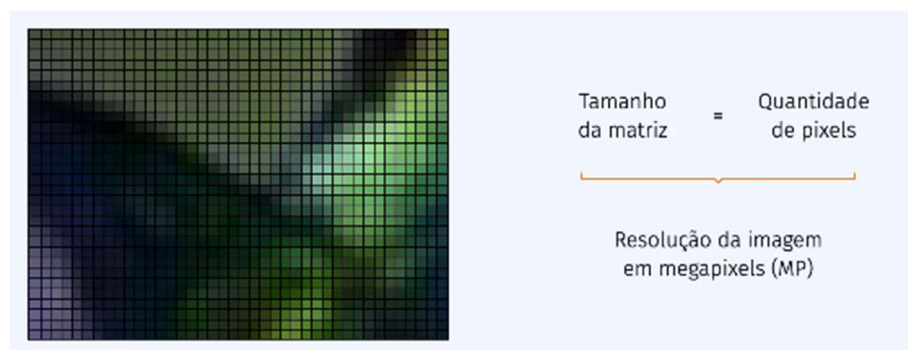
O modelo de cor mais comum hoje em dia temos o RGB (Red, Green, Blue).



Cada pixel é representado por três valores numéricos, uma para cada cor primária. Misturando esses dois valores conseguimos representar praticamente qualquer cor visível.



O retomando a definição de uma imagem digital, podemos dizer que ela é na verdade, uma matriz de pixels. O tamanho dessa matriz, ou seja, sua quantidade de pixel determina a resolução da imagem, geralmente em megapixels.



Como uma imagem é uma matriz podemos manipulá-las através de equações matemáticas. Podemos somar, dividir, filtrar e transformar a matriz para alterar a imagem.

Como podemos manipular uma imagem digital?

Podemos usar a biblioteca NumPy do Python que trás estrutura de dados e funções prontas que auxiliam e facilitam a manipulação de matrizes e vetores. A NumPy permite a execução de cálculos complexos de forma simples e eficiente.

Exemplo de uso da NumPy em uma imagem:

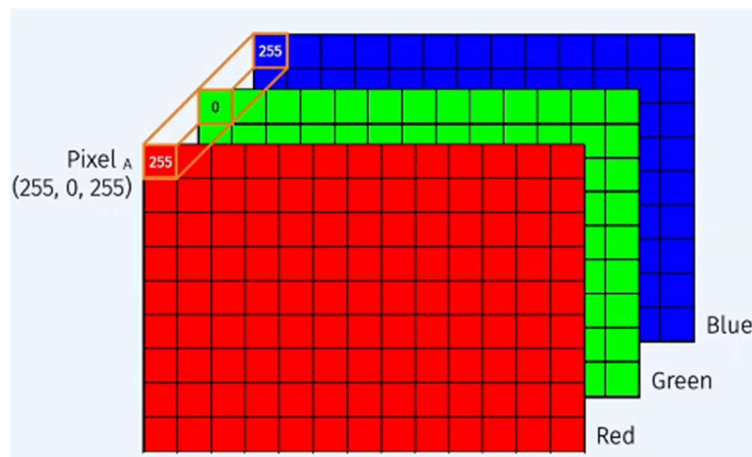
Deteção de cor predominante: Para isso, podemos contar quantas vezes cada uma das cores aparecem na imagem. A cor predominante irá aparecer mais vezes na contagem. Com a Numpy essas operações podem ser feita por linhas, colunas ou regiões específicas da imagem.

Aula 3 – Visualização de imagens

Se temos nossa imagem em forma de matriz como podemos ver o que estamos fazendo?

Para isso, podemos usar a biblioteca **Matplotlib**, uma ferramenta bastante utilizada em Python para visualização de dados. Neste curso, ela será aplicada principalmente para exibir imagens digitais, permitindo representar uma matriz numérica como uma imagem visível na tela.

Uma imagem é uma matriz de pixels, cada célula dessa matriz representa um ponto(pixels) da imagem, podendo ter um ou mais valores dependendo da escala de cor, que pode ser em preto e branco, RGB...



Aula 4 – Introdução a OpenCV

OpenCV é uma biblioteca de código aberto (por isso tem Open em seu nome), desenvolvida pela comunidade (profissionais e pesquisadores da área da visão computacional) e o CV vêm de Computer Vision. A OpenCV usa a estrutura de dados do Numpy, ou seja, imagens da OpenCV são vetores multidimensionais (matrizes em três dimensões – Altura, largura e comprimento) do NumPy. Usando essas duas bibliotecas (OpenCV e NumPy) em conjunto é possível a realização de operações matemáticas, filtros e transformações de forma rápida e eficiente.

Principais funções da OpenCV:

- Ler e salvar imagens em diferentes formatos;
- Redimensionar imagens;
- Capturar dados direto de uma câmera, como uma webcam;
- Aplicar transformações básicas de forma simples.

Escala de cores da OpenCV:

A biblioteca OpenCV não lê imagens no espaço de cor RGB e sim no BGR. Já a matplotlib por padrão usa a RGB. Para não ocorrer problemas de divergências de cores a OpenCV tem comandos próprios:

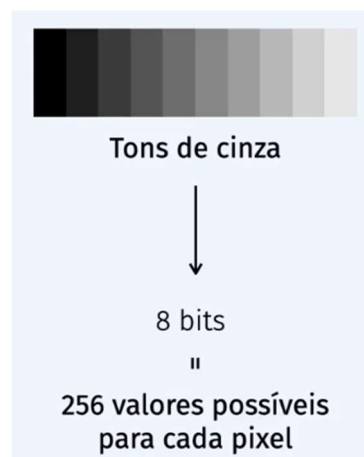
```
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
gain_rgb = np.clip(image_rgb / 255 * 1.001, 0, 1)
plt.imshow(gain_rgb)
```

Aula 5 – Operações básicas em Imagens

- Rotação e translação de imagens;
- Anotar ou desenhar polígonos nas imagens para validar localização;
- Transformações lineares;

Aula 6 – Conceitos fundamentais de imagens digitais

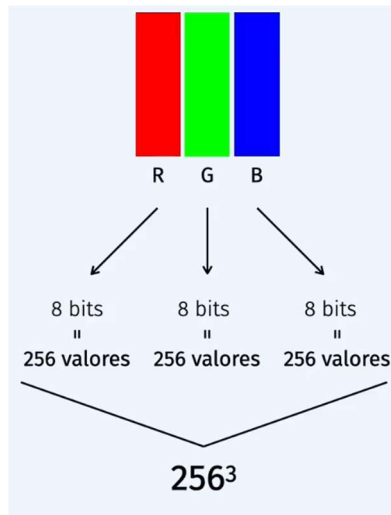
Como já explicados, imagens digitais são compostas por pixels, sendo o pixel o menor elemento de uma imagem. Além disso, uma imagem pode ser representada por uma matriz em que cada posição guarda o valor de um pixel. A palavra *pixel* vem da palavra *Picture Element* do inglês e cada pixel possui uma precisão. Em uma imagem nos tons de cinza temos para uma precisão de 8 bits $2^8 = 256$ possíveis valores para cada pixel. Ou seja, podemos representar 256 possíveis cores com uma precisão de 8 bits



Isso porque cada bit pode assumir dois valores, 1 ou 0.



Para o espaço de cores RGB temos três cores, 8 bits para cada uma delas, por tanto, $256^3 = 167\,777\,216$ de cores possíveis.



Resolução:

Quando falamos de dispositivos de reproduções como monitores, televisão, câmeras, projetores associamos suas resoluções a unidade de Megapixels, sendo mega $10^6 = 1\,000\,000$ (um milhão). Por exemplo uma câmera de 12 megapixels captura 12 milhões de pixels por imagem. Já quando nos referimos a uma televisão de 4K temos:



Largura = 3840 pixels

Altura = 2160 pixels

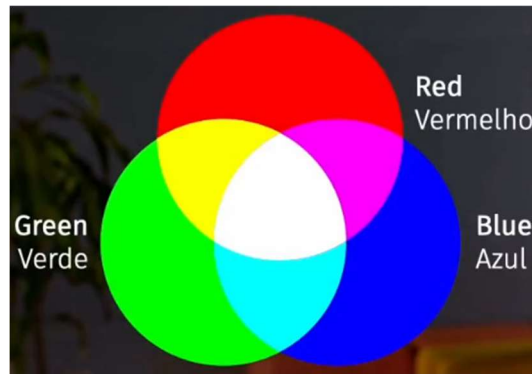
A quantidade total de pixels é:

$$3840 \times 2160 = 8.294.400 \text{ pixels}$$

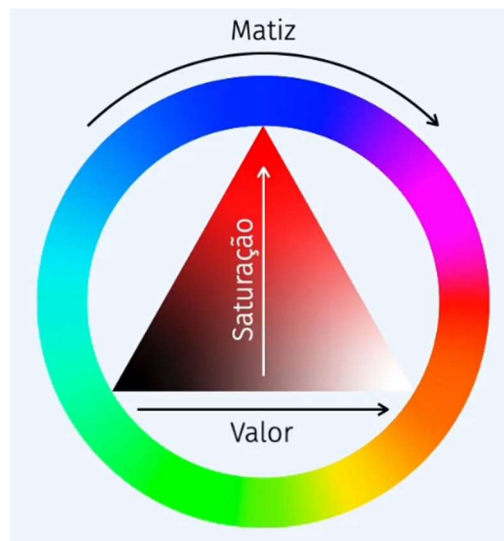
Então uma tela 4K tem aproximadamente 8,3 milhões de pixels.

Espaços de cores:

Já falamos do RGB:



Mas também existe o HSV:



Onde:

Matiz, representa a cor.

Saturação, mostra o quanto essa cor é intensa.

Valor, representa o brilho.

Uma cor pouco saturada terá uma aparência de tom cinza, aumentando-se o valor a cor torna-se mais brilhante. No espaço de cores RGB teríamos que alterar três valores para cada cor enquanto no HSV só precisamos alterar no máximo os três parâmetros: matiz, saturação, valor. Por esse motivo em tarefas de detecção de cores específicas o HSV ser mais usado que o RGB. Mas como vimos nas aulas práticos o OpenCV faz essas transformações de espaço de cores com comandos simples.

Binarização

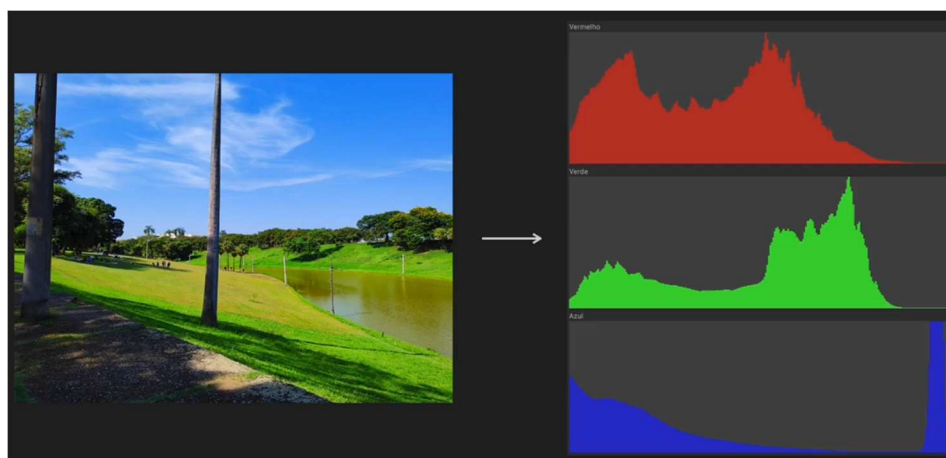
Em visão computacional é comum a necessidade de analisar distribuição de cores ou dos valores de pixels nos espaços de cores, isso permite selecionar cores de objetos de interesse. Esse processo gera um mapa binário tornando possível

identificar cores existentes e não existentes uma determinada imagem. Esse processo é chamado de binarização de imagens, sendo a segmentação de cores uma das formas de fazer a binarização.

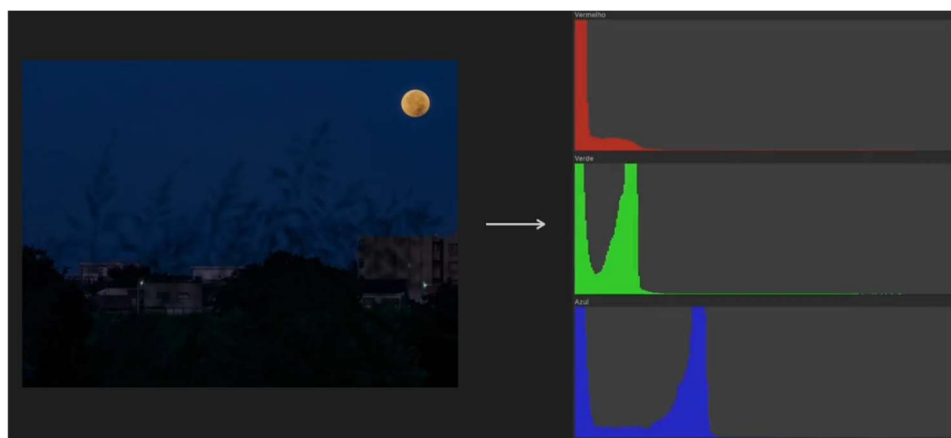
Histogramas

Outra forma de análise de imagem pode ser feita pelo histograma, sendo este um tipo de gráfico frequência que mostra quantas vezes cada valor de pixel aparece em uma determinada imagem.

Em uma foto do céu em um dia ensolarado, a contagem de cores dos tons de azul aparecem muito mais do que tons de amarelo.

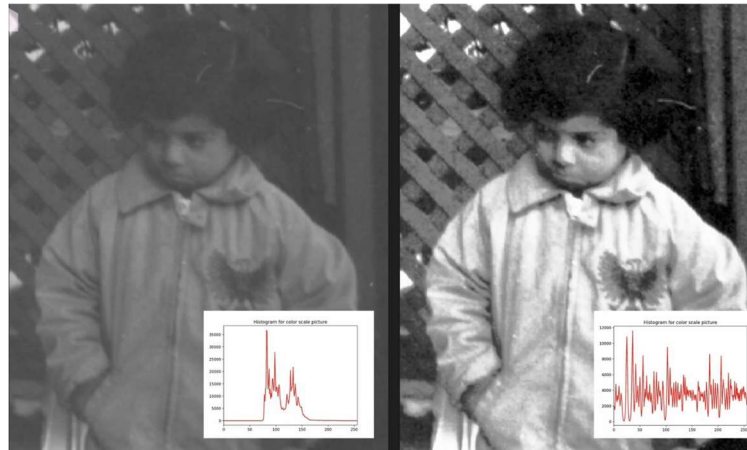


Já em uma foto de noite com poucas estrelas, haverá uma contagem baixa de tons de azul mas uma alta contagem de tons preto e amarelo.



Equalização de histograma

Técnica que ajusta o contraste da imagem para realçar detalhes trazendo maior contraste.



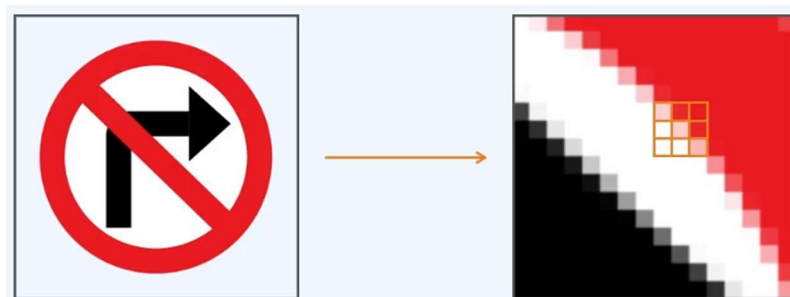
HoG – Histogram of Gradients

O histograma HoG é usado para detecção de objetos e neste caso não é feito baseado em relação as cores e sim sobre os gradientes que analisam as direções onde ocorrem mudança de brilho na imagem. Os gradientes representam vetores de direção onde há maior crescimento de um valor. A biblioteca OpenCV implementa tanto a Equalização de Histograma como o HoG.

Aula 7 – Fundamentos de filtros espaciais e convoluções

Essas ferramentas são muito úteis quando queremos realizar algum pré-processamento ou extrair informações relevantes de uma imagem.

As imagens contêm informações visuais e essa informação é espacial. Por exemplo, pensem em uma imagem de uma placa de trânsito, essa imagem traz informações sobre direções permitidas ou manobras proibidas. Mas essa informação só é transmitida quando um subconjunto de pixels tem uma determinada relação espacial. Eles formam um padrão, sendo está a relação espacial que carrega o significado visual.



Convolução:

Uma das formas de analisar a organização espacial dos pixels é através de convolução. A convolução é uma operação linear que relaciona dois sinais, esses sinais podem ter quaisquer dimensões que queiramos. Ela nos permite analisar e

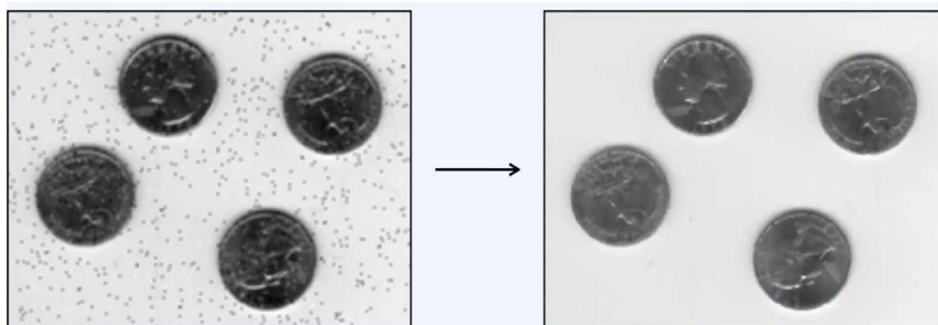
transformar a imagem a partir de relações espaciais. Para realizar uma convolução precisamos de duas funções, ou em outras palavras, dois sinais. O primeiro sinal é a imagem em si. O segundo sinal vai depender do tipo de informação da imagem que queremos extrair. Por exemplo, se quisermos detectar bordas, usamos um tipo de sinal específico.



Se quisermos aumentar ou reduzir a nitidez (sharpness), usamos outro tipo de sinal.



Se quisermos reduzir o ruído de uma imagem, usamos outro sinal específico.

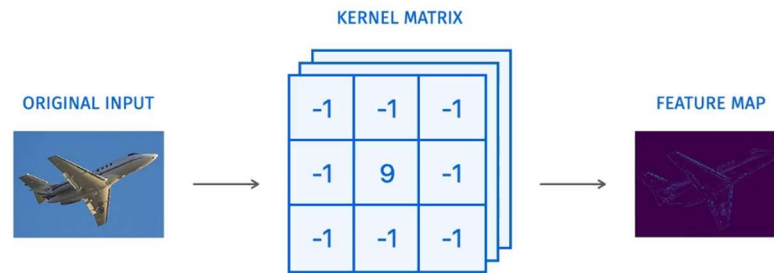


Cada variação desse segundo sinal altera o resultado da convolução.

Kernel:

O segundo sinal citado anteriormente é chamado de Kernel. Uma forma simples de entender isso é lembrar que uma imagem é uma matriz de pixel. O Kernel também é uma matriz, geralmente bem menor, que usamos para analisar ou modificar regiões específicas da imagem. Sendo, portanto, o filtro que extrai

informações espaciais, ou seja, informações sobre como os pixels se relacionam uns com os outros.



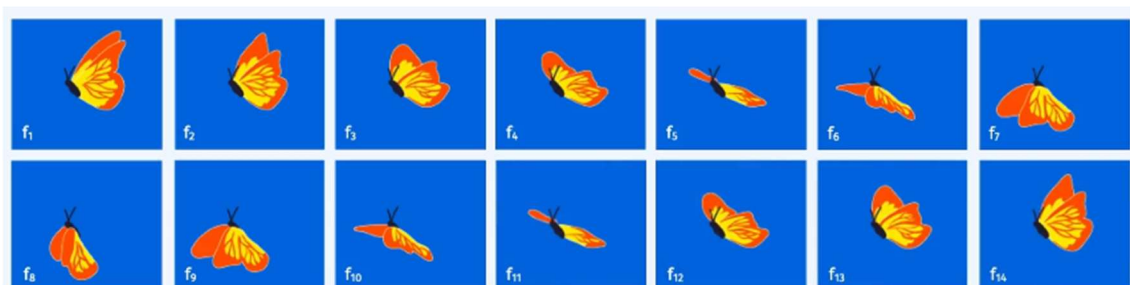
O Kernel é um segundo sinal que permite extrair informações espaciais da imagem, e existem vários tipos deles. Por exemplo, os kernels derivativos são usados para encontrar regiões de máxima variação nas imagens, o que na prática corresponde às bordas dos elementos na imagem.

As Redes Neurais Convolucionais (CNN), recebem esse nome justamente por causa da operação de convolução. Os parâmetros aprendidos durante o treinamento dessas redes são na verdade, os kernels.

Aula 8 – Introdução a processamento de vídeo

A maioria dos problemas reais do mundo não acontecem em imagens estáticas, por exemplo, um carro autônomo precisa processar vídeos em tempo real para entender o ambiente, identificar pedestres, semáforos, obstáculos e tudo isso enquanto o veículo está em movimento e compartilhando a via com outras pessoas.

O processamento de vídeo na prática é imaginar um vídeo como uma sequência de imagens (frames). O processo de processamento de vídeo acontece analisando cada frame individualmente. O desafio é que tudo precisa acontecer muito rápido isso requer que os algoritmos e modelos sejam eficientes para manter o funcionamento do sistema em tempo real.



Por exemplo, um vídeo a 30 fps (frames por segundo), tem 30 imagens por segundo. Isso significa que o sistema precisa processar cada imagem em menos de 33 milissegundos, exigindo que os softwares e hardware envolvidos sejam eficientes.

Além dos carros autônomos podemos exemplificar processamento de vídeos em filtros para vídeos chamadas tornando possível a remoção de fundo e alteração do cenário atrás do usuário. O processamento de vídeos também está presente na automação industrial, um sistema de monitoramento pode ser implementado junta a uma esteira fazendo a seleção de objetos ou produtos com base na cor, tamanho ou formato. Essa implementação pode ser feita desde encomendas em um centro de distribuição, até a seleção de grãos ou legumes de melhor qualidade em diferentes direções. Esses são apenas alguns dos muitos exemplos de aplicações da visão computacional.